Evaluación del módulo #6, #7 y #8

Consigna del proyecto 👨🏻‍💻

alkemy

Evaluación del módulo

Proyecto: Node & Express Web App

Situación inicial 📍

Unidad solicitante: Departamento de Desarrollo Backend de una empresa de tecnología.

Una reconocida empresa tecnológica necesita **desarrollar una aplicación web** para **gestión de usuarios y datos**. El equipo de arquitectura ha definido las bases del proyecto, pero requiere un **desarrollador backend** para llevarlo a producción.

Has sido seleccionado para este desafío. Tu misión será construir, iterar y demostrar el funcionamiento de una aplicación real utilizando **Node.js** y **Express**, aplicando conocimientos adquiridos en los **módulos 6, 7 y 8** del programa.

El proyecto deberá abordar, en **tres etapas**, los aspectos clave del desarrollo backend moderno: desde servir contenido web y gestionar rutas, hasta conectar con una base de datos, implementar un ORM, proteger rutas con JWT y exponer una API RESTful profesional.

Nuestro objetivo 📋

Desarrollar de manera individual una **aplicación web funcional** con **Node.js** y **Express**, que:

- Sirva contenido web dinámico.
- Gestione usuarios y datos persistidos en base de datos.
- Exponga una API RESTful con rutas protegidas mediante **JWT**.
- Integre persistencia con archivos planos, ORM para manejo de relaciones y funcionalidades clave como subida de archivos.

La implementación será **progresiva**, en tres partes alineadas a los módulos del curso:

→ **Módulo 6**: estructura inicial del servidor, rutas, vistas y persistencia básica.

→ **Módulo 7**: integración con base de datos, modelos y operaciones CRUD usando ORM.

→ **Módulo 8**: diseño de una API RESTful segura, con subida de archivos y autenticación.

Requerimientos ✅

Estos son los fundamentos que deberás respetar y aplicar durante todo el desarrollo del proyecto. Funcionan como condiciones base:

- La aplicación deberá estar desarrollada en **Node.js** (v18 o superior) utilizando el framework **Express.js**.
- El proyecto debe estar estructurado en **rutas**, **controladores**, **middlewares** y **servicios** siguiendo una arquitectura modular.
- Se espera el uso de **Sequelize** o **Mongoose** como ORM, según el tipo de base de datos seleccionada.
- La persistencia debe implementarse sobre una base de datos relacional (**PostgreSQL**) o documental (**MongoDB**).
- El repositorio del proyecto debe estar subido a **GitHub**, con estructura clara y un archivo `README.md` que documente su uso e instalación.
- Se deberá incluir al menos un mecanismo básico de seguridad utilizando **JSON Web Tokens (JWT)**.

Requerimientos específicos

Estos son los **elementos concretos** que la aplicación desarrollada deberá contemplar **funcional y técnicamente**. Serán validados en cada una de las entregas.

→ Gestión general de la aplicación

- Servir contenido web estático o dinámico desde **Express.**
- Implementar **rutas públicas y privadas** (protegidas con JWT).
- Permitir **autenticación de usuarios** (login / registro).
- Aplicar **lógica de persistencia** en archivos planos para funcionalidades simples (logs, configuración, etc.).
- Permitir **ejecución del servidor desde línea de comandos** con nodemon o scripts.

→ Acceso y gestión de datos

- Conectar la aplicación a una **base de datos real** (MongoDB o PostgreSQL).
- **Modelar y relacionar entidades** utilizando un ORM (relaciones 1:1, 1:N y N:M).
- Permitir **operaciones CRUD completas** sobre al menos dos entidades clave.
- Implementar **consultas filtradas y búsquedas** dinámicas.
- Aplicar **manejo de errores y validaciones** en las operaciones con base de datos.

→ Seguridad y API

- Crear una **API RESTful** con rutas protegidas mediante autenticación JWT.
- Implementar **subida de archivos** (por ejemplo, imágenes de usuario).
- Validar **tipos de archivo y tamaño** en el endpoint de subida.
- Estructurar las respuestas de la API con formato consistente (`status, message, data`).

Paso a paso 👣

Parte 2 – Módulo 7: *Acceso a datos en aplicaciones Node*

Este proyecto será avanzado paso a paso en cada clase y podrás completarlo progresivamente a través de los espacios ***asincrónicos.*** También tendrás espacios de consulta en las clases sincrónicas para despejar tus dudas.

En esta segunda parte, darás un paso clave en la evolución del backend: *conectar tu servidor con una base de datos*. La app ya no solo responderá rutas, sino que podrá **guardar, recuperar, modificar y eliminar datos reales de usuarios**. Este módulo te prepara para trabajar con bases de datos SQL usando tanto consultas tradicionales como herramientas modernas de abstracción (ORM). Además, te vas a enfrentar a desafíos reales como el manejo de relaciones y la necesidad de mantener consistencia con transacciones.

El objetivo será construir la conexión entre el servidor Express y una base de datos relacional, permitiendo la creación, lectura, actualización y eliminación de datos (CRUD). Se espera que apliques buenas prácticas en la consulta, protección e integración de datos, preparando el backend para exponer datos mediante API en la siguiente etapa.

1. **Conexión a una base de datos** (Lección 1)

   - Objetivo específico: Configurar una conexión estable y segura entre el servidor Node y una base de datos relacional (MySQL o PostgreSQL).

   - Tareas:
     → Crear la base de datos y al menos **1 tabla principal** (`usuarios` o equivalente).
     → Utilizar `mysql2, pg` o el paquete ORM elegido para establecer la conexión.
     → Almacenar credenciales en variables de entorno.

   - Requerimientos mínimos:

     → Archivo .env con las claves ocultas.
     → Log en consola al conectar con éxito.

   - Justificación esperada:

     → ¿Por qué elegiste ese cliente de conexión?

→ ¿Cómo se protegen los datos sensibles?

2. **Obtención de información desde una base de datos** (Lección 2)

- Objetivo específico: Consultar información almacenada y presentarla desde rutas del backend.
- Tareas:
  - → **Crear una ruta** `GET /usuarios` que devuelva los datos de la tabla.
  - → Procesar los resultados antes de enviarlos (evitar contraseñas o datos sensibles).
  - → Validar errores de conexión o consulta.
- Requerimientos mínimos:
  - → Al menos **3 registros simulados**.
  - → Respuesta en JSON clara y ordenada.
- Tarea PLUS (opcional):
  - → Implementar paginación o filtrado por query params (`?nombre=Juan`).

3. **Modificación de datos en una base de datos** (Lección 3)

- Objetivo específico: Incorporar operaciones de actualización y eliminación controladas sobre los datos existentes.
- Tareas:
  - → Ruta `PUT /usuarios/:id` para modificar un registro.
  - → Ruta `DELETE /usuarios/:id` con validación previa de existencia.
  - → Validar errores y devolver mensajes útiles.
- Requerimientos mínimos:
  - → Confirmación de éxito en ambas operaciones.
  - → Validación de ID existente.
- Justificación esperada:
  - → ¿Por qué decidiste actualizar sólo ciertos campos?

→ ¿Qué validaciones aplicaste para evitar errores?

4. **Transaccionalidad** (Lección 4)

- Objetivo específico: Proteger operaciones sensibles asegurando consistencia de datos.
- Tareas:
  - → Implementar una operación simulada que involucre al menos 2 acciones consecutivas (por ejemplo, registrar un usuario y crear su historial).
  - → Asegurar rollback si alguna falla.
- Requerimientos mínimos:
  - → Log de éxito o error claro.
  - → Evidencia de rollback si se fuerza un error.
- Tarea PLUS (opcional):
  - → Log en archivo de las transacciones fallidas (similar al `log.txt` previo).

5. **Acceso a datos con ORM** (Lección 5)

- Objetivo específico: Reemplazar o complementar las consultas SQL con un ORM (como Sequelize).
- Tareas:
  - → Instalar e inicializar ORM.
  - → Definir al menos 1 modelo (`User`).
  - → Crear una ruta que devuelva los usuarios usando métodos del ORM.
- Requerimientos mínimos:
  - → Comparación de resultados entre SQL manual y ORM.
- Justificación esperada:
  - → ¿Qué ventaja encontraste usando ORM frente al cliente SQL tradicional?

6. **Manejo de relaciones en un ORM** (Lección 6)

   - Objetivo específico: Crear relaciones entre modelos y consultarlas desde rutas.
   - Tareas:
     → Crear al menos 1 relación (por ejemplo, `Usuario` tiene muchos `Pedidos`).
     → Crear una ruta que devuelva el usuario y sus pedidos en una sola consulta.
   - Requerimientos mínimos:
     → Uso de `include` o equivalente para traer relaciones.
     → Al menos 2 modelos relacionados.
   - Tarea PLUS (opcional):
     → Mostrar los datos anidados en una tabla en HTML o como JSON ordenado.

✅ Entregables

1. **Repositorio actualizado en GitHub**
   - Nuevos modelos y rutas.
   - Carpeta `models/, services/` o equivalente con acceso a datos.
   - Código versionado y documentado.
2. **Subcarpeta nueva en Google Drive**
   - Carpeta: *Parte 2 – Módulo 7*
   - Capturas de las operaciones de lectura, escritura y eliminación desde Postman o similar.
   - (Opcional) Documento `.md` con breve reflexión sobre decisiones técnicas.

Recuerda mantener actualizada la carpeta general del proyecto y organizar bien cada entrega por módulo.

🔍 ¿Qué vamos a validar?

Durante la evaluación de esta segunda entrega, se verificará que el backend esté correctamente conectado a una base de datos y sea capaz de realizar operaciones completas de acceso y modificación de datos. Evaluaremos los siguientes aspectos:

- ★ Aplicación de requerimientos técnicos
  - ☐ Conexión estable y segura a una base de datos relacional.
  - ☐ Creación de tablas y estructuras acordes a los datos manipulados.
  - ☐ Implementación funcional de rutas `GET`, `POST`, `PUT` y `DELETE`.
  - ☐ Manejo adecuado de errores y validaciones.
  - ☐ Uso correcto de un ORM para consultas y relaciones (cuando aplique).
- ★ Objetivo de la consigna
  - ☐ Se espera que la aplicación logre implementar un CRUD completo sobre datos reales, con lógica consistente.
  - ☐ Se valorará el uso de relaciones, modularización del acceso a datos y coherencia con la arquitectura del proyecto.
  - ☐ Se validará que esta parte del backend quede lista para exponer los datos como una API REST en el próximo módulo.
- ★ Justificación y reflexión
  - ☐ Claridad en la elección de herramientas y estrategias para manejar datos.
  - ☐ Documentación en código o README sobre modelos, relaciones y transacciones.
  - ☐ Explicación del uso o no uso de ORM, estructura de carpetas, validaciones, etc.
- ★ Claridad y organización
  - ☐ Orden lógico del código, separación por módulos, nombres representativos.
  - ☐ Comentarios que expliquen lo esencial.
  - ☐ Instrucciones en el `README.md` claras para ejecutar esta segunda parte.

💼 Portafolio

Esta entrega representa tu capacidad para conectar una aplicación real con una base de datos, uno de los pilares más importantes del desarrollo backend. Incluir esta parte en tu portafolio puede ayudarte a mostrar:

- Conocimiento práctico del stack Node + SQL.
- Capacidad para implementar un CRUD completo con buenas prácticas.
- Dominio de herramientas modernas como ORM y transacciones.

¡Éxitos!

Nos vemos más adelante

alkemy